

FIAP

Análise e Desenvolvimento de Sistemas

Sistema Fintech

Fase 6 — Comandos SQL para manipulação de dados (Oracle)

Larissa Gomes de Carvalho
RM 571266 · Turma 1TDSOA
São Paulo, setembro de 2026

1. Introdução

Este documento reúne os comandos SQL que o sistema **Fintech** — aplicação Java de gestão financeira pessoal — utilizará para manipular os dados no banco **Oracle**: cadastro, alteração, consultas simples, consultas ordenadas e a consulta do dashboard.

Os comandos foram escritos sobre o modelo relacional criado na Fase 3, evoluído conforme permitido no enunciado ("podendo adaptá-la ou evolui-la conforme as necessidades do seu projeto"). As evoluções estão listadas na seção 2.3.

Todos os comandos foram **executados e validados em um banco Oracle** com uma massa de dados de teste antes da entrega (50 de 50 casos de teste aprovados — detalhes no Anexo A).

2. Modelo relacional

Diagramas do modelo relacional do Sistema Fintech (prefixo T_SF_). A tabela T_SF_INVESTIMENTO foi incluída nesta fase para atender às operações de investimento solicitadas.

2.1 Modelo lógico

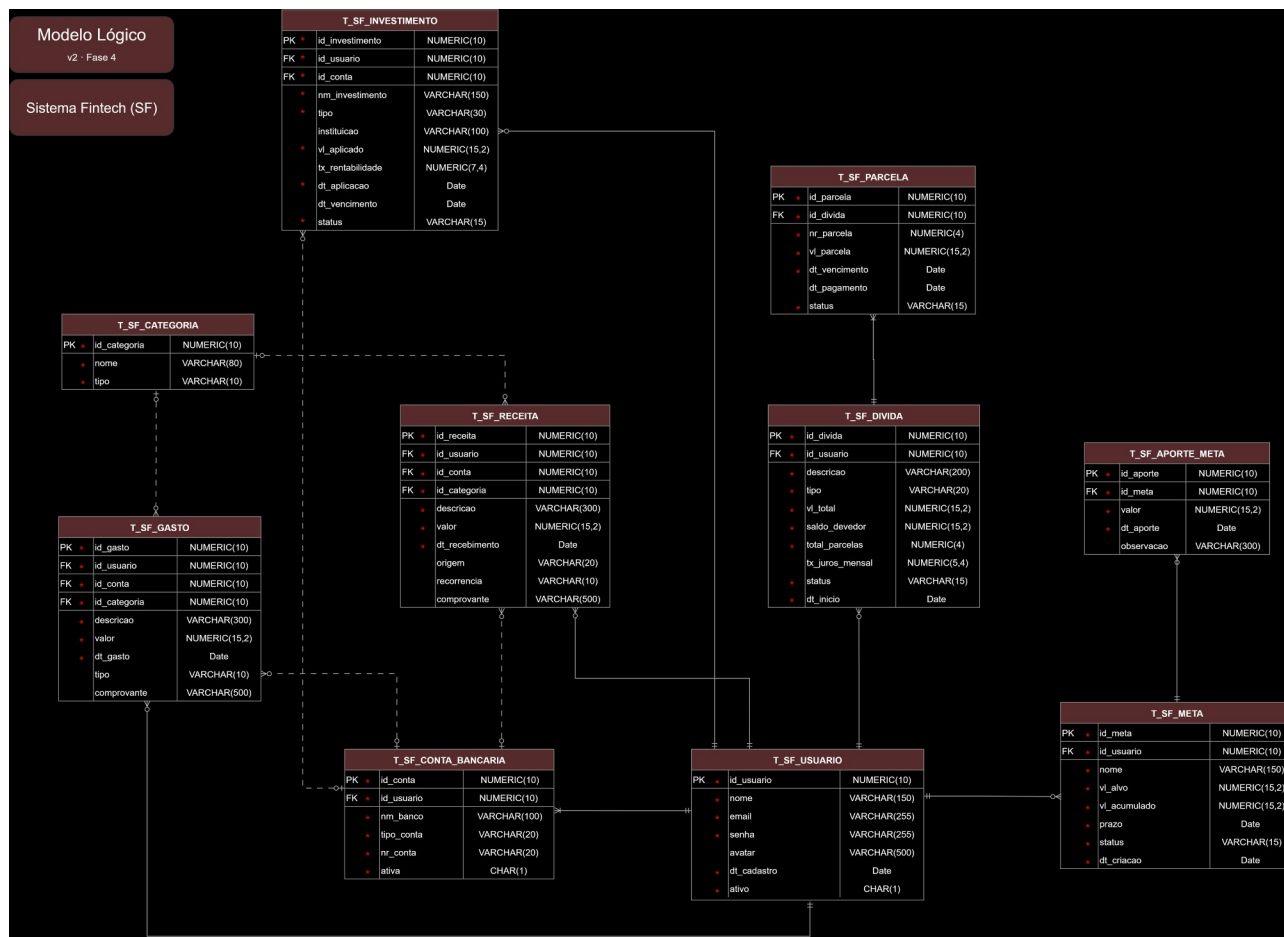


Figura 2.1 — Modelo lógico (v2, Fase 6). Asterisco vermelho = coluna obrigatória.

2.2 Modelo físico

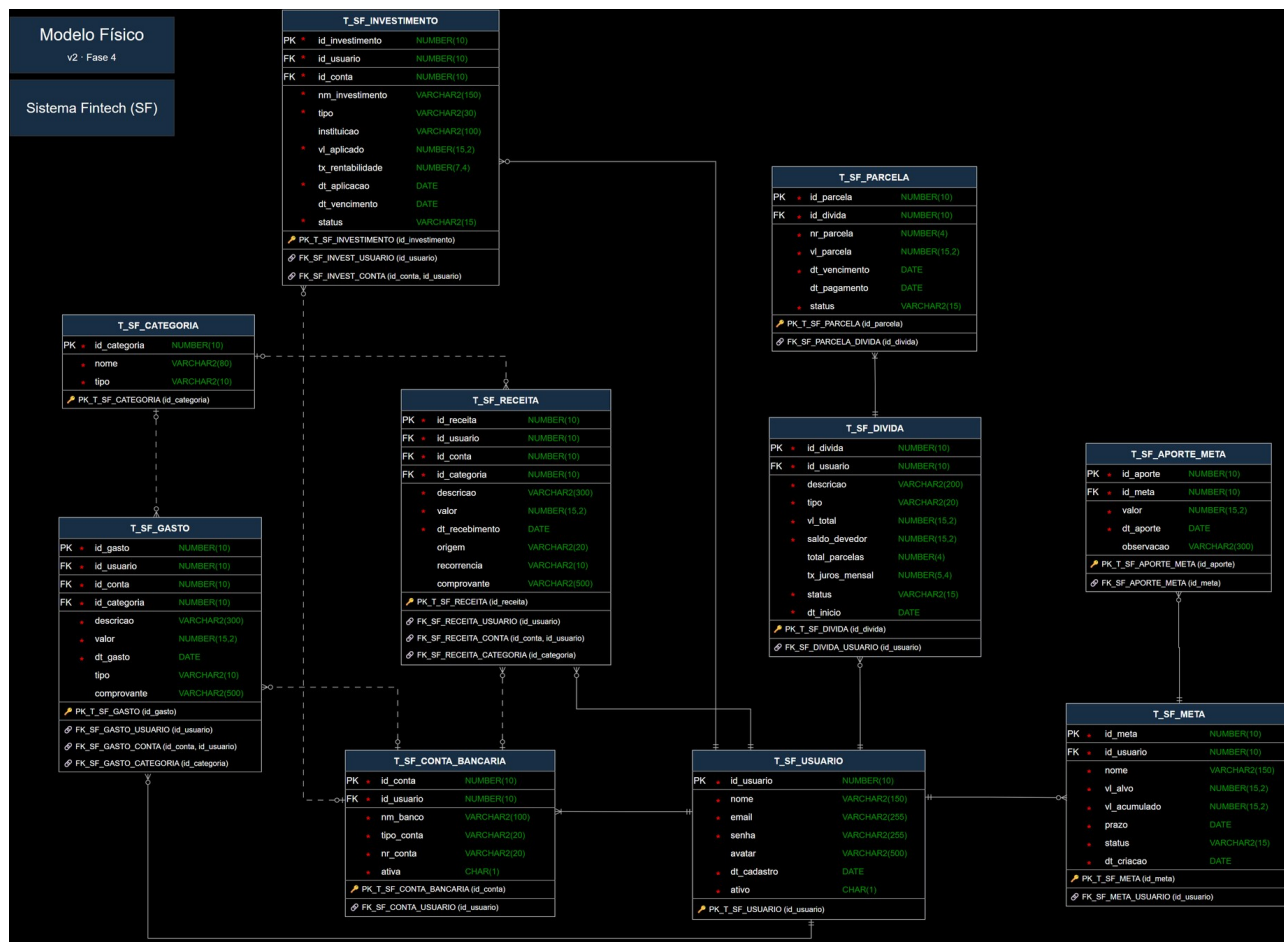


Figura 2.2 — Modelo físico (v2, Fase 6). Asterisco vermelho = coluna obrigatória.

2.3 Evoluções em relação à Fase 3

Evolução	Motivo
Nova tabela T_SF_INVESTIMENTO	O enunciado exige cadastro, alteração e consulta de investimentos, conceito ausente no modelo da Fase 3.
Chaves primárias com IDENTITY	O banco gera os códigos; por isso os INSERT não informam a coluna de ID, como nos exemplos do enunciado.
Domínios com CHECK	Garante valores válidos (ex.: tipo do gasto FIXO ou VARIÁVEL).
E-mail único	Impede dois usuários com o mesmo e-mail (User Story 1).
FK composta (id_conta, id_usuario)	Garante que a conta usada em receita, gasto ou investimento pertence ao mesmo usuário.
dt_pagamento da parcela opcional	Permite registrar parcelas em aberto ou em atraso (User Story 4).
Taxas como fração decimal	0.0299 = 2,99%; mantém o tipo NUMBER(5,4) da Fase 3 comportando juros de cartão acima de 10% a.m.

Nenhuma coluna da Fase 3 foi removida ou renomeada.

3. Convenção das máscaras de substituição

Os valores dinâmicos, que a aplicação Java preencherá, aparecem como máscaras:

Tipo do valor	Máscara	Exemplo
Texto	'[NOME DO CAMPO]'	'[DESCRIÇÃO DO GASTO]'
Numérico	[NOME DO CAMPO]	[VALOR DO GASTO]
Data	TO_DATE('[DATA]', 'DD/MM/YYYY')	TO_DATE('[DATA DO GASTO]', 'DD/MM/YYYY')

Valores de domínio são informados em maiúsculas e sem acento (ex.: 'VARIABEL'). Os comandos não contêm COMMIT: o controle da transação fica com a aplicação.

4. Comandos SQL

4.1 Cadastro

4.1.1 Cadastrar os dados de um novo usuário

```
INSERT INTO T_SF_USUARIO (nome, email, senha, avatar, dt_cadastro, ativo)
VALUES ('[NOME DO USUÁRIO]', '[E-MAIL DO USUÁRIO]', '[HASH DA SENHA]',
        '[CAMINHO DO AVATAR]', SYSDATE, 'S');
```

4.1.2 Cadastrar os dados da conta bancária de um usuário

```
INSERT INTO T_SF_CONTA_BANCARIA (id_usuario, nm_banco, tipo_conta, nr_conta,
                                  ativa)
VALUES ([CÓDIGO DO USUÁRIO], '[NOME DO BANCO]', '[TIPO DA CONTA]',
        '[NÚMERO DA CONTA]', 'S');
```

4.1.3 Cadastrar os dados de uma nova receita (entrada de dinheiro) para um usuário

```
INSERT INTO T_SF_RECEITA (id_usuario, id_conta, id_categoria, descricao,
                           valor, dt_recebimento, origem, recorrencia,
                           comprovante)
VALUES ([CÓDIGO DO USUÁRIO], [CÓDIGO DA CONTA], [CÓDIGO DA CATEGORIA],
        '[DESCRIÇÃO DA RECEITA]', [VALOR DA RECEITA],
        TO_DATE('[DATA DO RECEBIMENTO]', 'DD/MM/YYYY'),
        '[ORIGEM DA RECEITA]', '[RECORRÊNCIA DA RECEITA]',
        '[CAMINHO DO COMPROVANTE]);
```

4.1.4 Cadastrar os dados de uma nova despesa (gasto) de um usuário

```
INSERT INTO T_SF_GASTO (id_usuario, id_conta, id_categoria, descricao, valor,
                        dt_gasto, tipo, comprovante)
VALUES ([CÓDIGO DO USUÁRIO], [CÓDIGO DA CONTA], [CÓDIGO DA CATEGORIA],
        '[DESCRIÇÃO DO GASTO]', [VALOR DO GASTO],
        TO_DATE('[DATA DO GASTO]', 'DD/MM/YYYY'),
        '[TIPO DO GASTO]', '[CAMINHO DO COMPROVANTE]);
```

4.1.5 Cadastrar os dados de um novo investimento feito por um usuário

```
INSERT INTO T_SF_INVESTIMENTO (id_usuario, id_conta, nm_investimento, tipo,
                                 instituicao, vl_aplicado, tx_rentabilidade,
                                 dt_aplicacao, dt_vencimento, status)
VALUES ([CÓDIGO DO USUÁRIO], [CÓDIGO DA CONTA], '[NOME DO INVESTIMENTO]',
        '[TIPO DO INVESTIMENTO]', '[INSTITUIÇÃO]', [VALOR APLICADO],
        [TAXA DE RENTABILIDADE],
        TO_DATE('[DATA DA APLICAÇÃO]', 'DD/MM/YYYY'),
        TO_DATE('[DATA DE VENCIMENTO]', 'DD/MM/YYYY'), 'ATIVO');
```

4.2 Alteração

4.2.1 Alterar os dados de um usuário (a partir do código do usuário)

```
UPDATE T_SF_USUARIO
SET nome = '[NOME DO USUÁRIO]',
    email = '[E-MAIL DO USUÁRIO]',
    avatar = '[CAMINHO DO AVATAR]'
WHERE id_usuario = [CÓDIGO DO USUÁRIO];
```

4.2.2 Alterar os dados de uma receita (a partir do código do usuário e do código da receita)

```
UPDATE T_SF_RECEITA
SET id_conta = [CÓDIGO DA CONTA],
    id_categoria = [CÓDIGO DA CATEGORIA],
    descricao = '[DESCRIÇÃO DA RECEITA]',
    valor = [VALOR DA RECEITA],
    dt_recebimento = TO_DATE('[DATA DO RECEBIMENTO]', 'DD/MM/YYYY'),
    origem = '[ORIGEM DA RECEITA]',
    recorrencia = '[RECORRÊNCIA DA RECEITA]',
    comprovante = '[CAMINHO DO COMPROVANTE]'
WHERE id_receita = [CÓDIGO DA RECEITA]
AND id_usuario = [CÓDIGO DO USUÁRIO];
```

4.2.3 Alterar os dados de uma despesa (a partir do código do usuário e do código da despesa)

```
UPDATE T_SF_GASTO
SET id_conta = [CÓDIGO DA CONTA],
    id_categoria = [CÓDIGO DA CATEGORIA],
    descricao = '[DESCRIÇÃO DO GASTO]',
    valor = [VALOR DO GASTO],
    dt_gasto = TO_DATE('[DATA DO GASTO]', 'DD/MM/YYYY'),
    tipo = '[TIPO DO GASTO]',
    comprovante = '[CAMINHO DO COMPROVANTE]'
WHERE id_gasto = [CÓDIGO DO GASTO]
AND id_usuario = [CÓDIGO DO USUÁRIO];
```

4.2.4 Alterar os dados de um investimento (a partir do código do usuário e do código do investimento)

```
UPDATE T_SF_INVESTIMENTO
SET id_conta = [CÓDIGO DA CONTA],
    nm_investimento = '[NOME DO INVESTIMENTO]',
    tipo = '[TIPO DO INVESTIMENTO]',
    instituicao = '[INSTITUIÇÃO]',
    vl_aplicado = [VALOR APLICADO],
    tx_rentabilidade = [TAXA DE RENTABILIDADE],
    dt_aplicacao = TO_DATE('[DATA DA APLICAÇÃO]', 'DD/MM/YYYY'),
    dt_vencimento = TO_DATE('[DATA DE VENCIMENTO]', 'DD/MM/YYYY'),
    status = '[STATUS DO INVESTIMENTO]'
WHERE id_investimento = [CÓDIGO DO INVESTIMENTO]
AND id_usuario = [CÓDIGO DO USUÁRIO];
```

4.3 Consultas simples

4.3.1 Consultar os dados de um usuário específico (usando o código dele)

```
SELECT id_usuario, nome, email, avatar, dt_cadastro, ativo
FROM T_SF_USUARIO
WHERE id_usuario = [CÓDIGO DO USUÁRIO];
```

4.3.2 Consultar os dados de uma despesa específica (usando o código do usuário e o da despesa)

```
SELECT id_gasto, id_usuario, id_conta, id_categoria, descricao, valor,
    dt_gasto, tipo, comprovante
FROM T_SF_GASTO
WHERE id_gasto = [CÓDIGO DO GASTO]
AND id_usuario = [CÓDIGO DO USUÁRIO];
```

4.3.3 Consultar os dados de um investimento específico (com código do usuário e do investimento)

```
SELECT id_investimento, id_usuario, id_conta, nm_investimento, tipo,
       instituicao, vl_aplicado, tx_rentabilidade, dt_aplicacao,
       dt_vencimento, status
FROM T_SF_INVESTIMENTO
WHERE id_investimento = [CÓDIGO DO INVESTIMENTO]
AND id_usuario = [CÓDIGO DO USUÁRIO];
```

4.4 Consultas ordenadas

4.4.1 Consultar todas as despesas de um usuário, ordenadas da mais recente à mais antiga

```
SELECT id_gasto, id_usuario, id_conta, id_categoria, descricao, valor,
       dt_gasto, tipo, comprovante
FROM T_SF_GASTO
WHERE id_usuario = [CÓDIGO DO USUÁRIO]
ORDER BY dt_gasto DESC, id_gasto DESC;
```

4.4.2 Consultar todos os investimentos de um usuário, do mais recente ao mais antigo

```
SELECT id_investimento, id_usuario, id_conta, nm_investimento, tipo,
       instituicao, vl_aplicado, tx_rentabilidade, dt_aplicacao,
       dt_vencimento, status
FROM T_SF_INVESTIMENTO
WHERE id_usuario = [CÓDIGO DO USUÁRIO]
ORDER BY dt_aplicacao DESC, id_investimento DESC;
```

4.5 Consulta para o dashboard

4.5.1 Consultar as informações principais de um usuário, junto com a última despesa e o último investimento deste usuário (retorna apenas uma linha)

```
SELECT u.id_usuario,
       u.nome,
       u.email,
       u.dt_cadastro,
       g.id_gasto AS id_ultimo_gasto,
       g.descricao AS descricao_ultimo_gasto,
       g.valor AS valor_ultimo_gasto,
       g.dt_gasto AS dt_ultimo_gasto,
       g.tipo AS tipo_ultimo_gasto,
       i.id_investimento AS id_ultimo_investimento,
       i.nm_investimento AS nm_ultimo_investimento,
       i.tipo AS tipo_ultimo_investimento,
       i.vl_aplicado AS vl_ultimo_investimento,
       i.dt_aplicacao AS dt_ultimo_investimento
FROM T_SF_USUARIO u
LEFT JOIN (SELECT id_usuario, id_gasto, descricao, valor, dt_gasto, tipo,
                 ROW_NUMBER() OVER (ORDER BY dt_gasto DESC,
                                     id_gasto DESC) AS rn
          FROM T_SF_GASTO
          WHERE id_usuario = [CÓDIGO DO USUÁRIO]) g
ON g.id_usuario = u.id_usuario
AND g.rn = 1
LEFT JOIN (SELECT id_usuario, id_investimento, nm_investimento, tipo,
                 vl_aplicado, dt_aplicacao,
                 ROW_NUMBER() OVER (ORDER BY dt_aplicacao DESC,
                                     id_investimento DESC) AS rn
          FROM T_SF_INVESTIMENTO
          WHERE id_usuario = [CÓDIGO DO USUÁRIO]) i
ON i.id_usuario = u.id_usuario
AND i.rn = 1
WHERE u.id_usuario = [CÓDIGO DO USUÁRIO];
```

Anexo A — Validação dos comandos

Os comandos da seção 4 foram executados em Oracle Database Free (container Docker), com sintaxe mantida compatível com o Oracle 19c. Para cada caso de teste, o comando entregue foi usado exatamente como está, apenas com as máscaras substituídas por valores de uma massa de dados de teste.

Grupo	Casos	O que foi verificado
Estrutura	6	10 tabelas, objetos válidos, constraints nomeadas, índices, IDENTITY
Cadastro	9	5 INSERT válidos; rejeição de e-mail duplicado, domínio inválido, usuário inexistente e conta de outro usuário
Alteração	7	cada UPDATE afeta só o registro do usuário informado; senha preservada
Consultas simples	5	registro correto; 0 linhas para outro usuário; senha nunca retornada
Consultas ordenadas	6	ordem da mais recente à mais antiga, com desempate por código
Dashboard	7	sempre 1 linha por usuário, inclusive sem despesas ou investimentos
Constraints e compatibilidade	10	regras de integridade e ausência de recursos exclusivos de versões recentes do Oracle

Resultado: 50 de 50 casos aprovados.